

课后作业

环境：IsaacSim+IsaacLab+MuJoCo 参考资料：

- [APLaS-Plus/unitree_rl_lab](#)
- [unitreerobotics/unitree_mujoco](#)
- [google-deepmind/mujoco:Multi-JointdynamicswithContact.Ageneralpurposephysicsimulator.](#)
- [unitreerobotics/unitree_sdk2:Unitreerobotsdkversion2.https://support.unitree.com/home/zh/developer](#)

基础作业

1.1 环境搭建与训练验证

- 确认 UnitreeRLLab 安装成功，运行以下命令列出所有可用任务：

```
cd~/unitree_rl_lab/./unitree_rl_lab.sh-l
```

- 启动 Unitree-G1-29dof-Velocity 训练：

```
./unitree_rl_lab.sh-t--taskUnitree-G1-29dof-Velocity
```

- 观察训练成功启动后，使用 Ctrl+C 结束训练；
- 找到本次训练生成的 run 目录，列出其中的文件。

提供以下验证材料：

- 任务列表截图 1 张：能看到 5 个可用任务；
- 训练启动截图 1 张：能看到训练输出和 reward 值；
- run 目录文件列表截图 1 张。

1.2 训练输出分析

结合训练终端输出、操作手册和课堂内容，回答以下问题：

- 对比第八讲的 Isaac-Humanoid-v0，Unitree-G1-29dof-Velocity 新增了哪些 reward 项？这些新增项分别面向什么需求？
- 训练输出中有两个 Curriculum 指标：Curriculum/terrain_levels 和 Curriculum/lin_vel_cmd_levels，它们各自含义是什么？为什么训练初期它们的值都很低？
- 训练初期 Episode_Termination/bad_orientation 的值为 1.0，这说明了什么？随

着训练推进，这个值应该发生什么变化？

1.3 奖励函数源码分析

操作手册中给出了 3 个奖励函数的源码 (upward、energy、foot_clearance_reward)，回答以下问题：

- 选择其中 1 个函数，用自己的话解释它的工作原理（输入是什么、计算逻辑是什么、输出代表什么）；
- foot_clearance_reward 中使用了 tanh 函数，它的作用是什么？为什么脚着地时不会误罚支撑脚？

1.4 Sim2Sim 概念理解

- 用一段话解释：什么是 Sim2Sim？为什么不直接把 IsaacLab 训练的模型部署到真机上，而要先在 MuJoCo 中验证？
- 用一段话解释：为什么 Sim2Sim 验证选择 MuJoCo 而不是 Gazebo？从物理引擎的接触模型、运行速度、API 设计等角度分析；
- ONNX 格式的作用是什么？为什么不直接用 PyTorch 的.pt 文件进行部署？

1.5 Aelos 机器人基础操作

- 阅读 SMART 基础使用教程，了解 Aelos 机器人的启动流程和安全注意事项；
- 安装示教软件，完成与机器人的连接；
- 启动机器人，通过示教软件完成至少 2 个简单的动作（如站立、挥手、蹲下等）；
- 提供以下验证材料：
 - 机器人启动截图 1 张；
 - 示教软件界面截图 1 张；
 - 机器人执行动作的照片。

1.6 Aelos 机器人自由度分析

- 分析 Aelos 的自由度 (DoF) 分布；
- 基于 Aelos 的自由度分析，思考以下问题：
 - 你想利用 Aelos 完成哪些类型的任务？举例说明；
 - 你觉得 Aelos 目前最难以完成的任务是什么？为什么？

基础作业提交要求：请将以上内容整理为一份 PDF，包含必要的截图说明和问题的回答。

Answers:

1.1 环境搭建与训练验证

(1) 任务列表

```
(isaac@lab) root@puffree-container:~$ vim ~/unitree_rl_lab/source/unitree_rl_lab/unitree_rl_lab/assets/robots/unitree.py
(isaac@lab) root@puffree-container:~$ cd unitree_rl_lab/
(isaac@lab) root@puffree-container:~/unitree_rl_lab$ ./unitree_rl_lab.sh -l
```

S. No.	Task Name	Entry Point	Config
1	Unitree-G1-29dof-Velocity	isaac@lab.envs:ManagerBasedRLEnv	locomotion.robots.g1.29dof.velocity_env_cfg:RobotEnvCfg
2	Unitree-Go2-Velocity	isaac@lab.envs:ManagerBasedRLEnv	locomotion.robots.go2.velocity_env_cfg:RobotEnvCfg
3	Unitree-H1-Velocity	isaac@lab.envs:ManagerBasedRLEnv	locomotion.robots.h1.velocity_env_cfg:RobotEnvCfg
4	Unitree-G1-29dof-Mimic-Dance-102	isaac@lab.envs:ManagerBasedRLEnv	mimic.robots.g1_29dof.dance_102.tracking_env_cfg:RobotEnvCfg
5	Unitree-G1-29dof-Mimic-Gangnam-Style	isaac@lab.envs:ManagerBasedRLEnv	mimic.robots.g1_29dof.gangnam_style.tracking_env_cfg:RobotEnvCfg

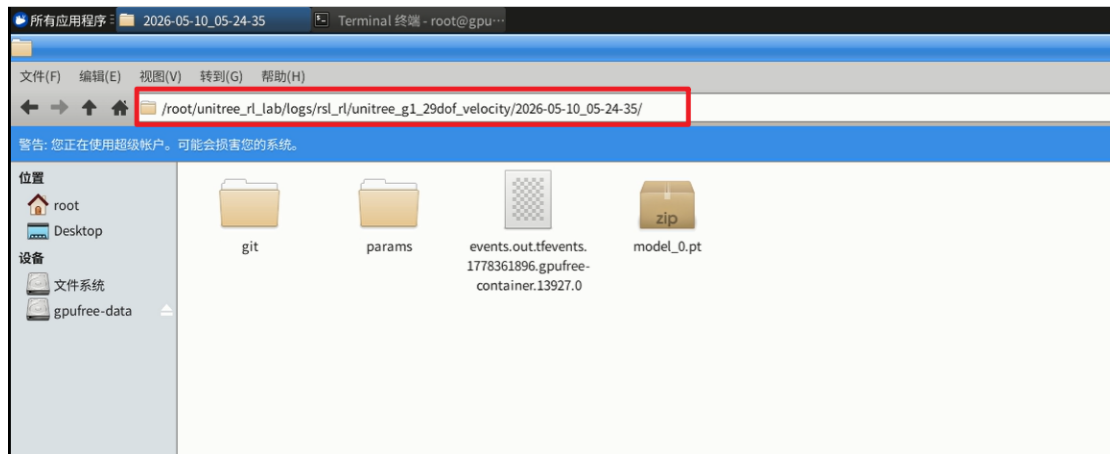
(2) 训练启动

```
Episode Termination/bad_orientation: 1.0000
-----
Total timesteps: 340640
Iteration time: 1.78s
Time elapsed: 00:01:05
ETA: 01:56:42

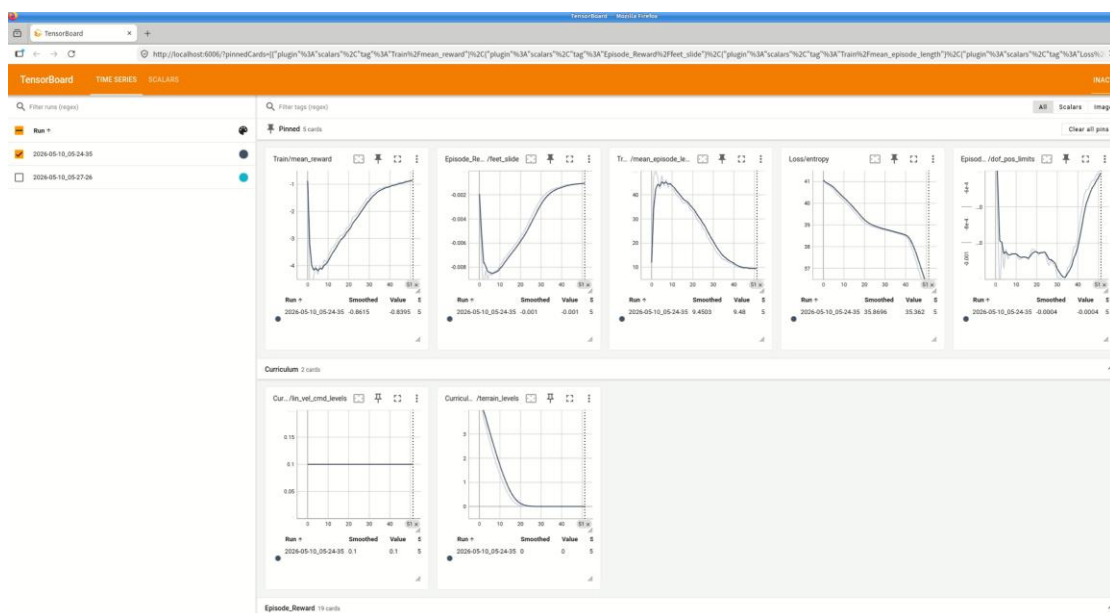
*****
Learning iteration 35/50000

Computation: 55677 steps/s (collection: 1.671s, learning 0.095s)
  Mean action noise std: 0.92
  Mean value function loss: 0.0089
  Mean surrogate loss: -0.0131
  Mean entropy loss: 38.6303
  Mean reward: -1.16
  Mean episode length: 13.08
  Episode_Reward/track_lin_vel_xy: 0.0065
  Episode_Reward/track_ang_vel_z: 0.0008
  Episode_Reward/alive: 0.0019
  Episode_Reward/base_linear_velocity: -0.0021
  Episode_Reward/base_angular_velocity: -0.0126
  Episode_Reward/joint_vel: -0.0097
  Episode_Reward/joint_acc: -0.0041
  Episode_Reward/action_rate: -0.0298
  Episode_Reward/dof_pos_limits: -0.0009
  Episode_Reward/energy: -0.0002
  Episode_Reward/joint_deviation_arms: -0.0022
  Episode_Reward/joint_deviation_waists: -0.0033
  Episode_Reward/joint_deviation_legs: -0.0049
  Episode_Reward/flat_orientation_l2: -0.0118
  Episode_Reward/base_height: -0.0000
  Episode_Reward/gait: 0.0030
  Episode_Reward/feet_slide: -0.0016
  Episode_Reward/feet_clearance: 0.0124
  Episode_Reward/undesired_contacts: -0.0002
  Curriculum/terrain_levels: 0.0000
  Curriculum/lin_vel_cmd_levels: 0.1000
  Metrics/base_velocity/error_vel_xy: 0.0107
  Metrics/base_velocity/error_vel_yaw: 0.0610
  Episode Termination/time out: 0.0000
  Episode Termination/base_height: 0.0000
  Episode Termination/bad_orientation: 1.0000
-----
Total timesteps: 3538944
Iteration time: 1.77s
Time elapsed: 00:01:07
ETA: 01:54:16
```

(3) 目录文件列表



(4) TensorBoard



1.2 训练输出分析

- 对比第八讲的 Isaac-Humanoid-v0, Unitree-G1-29dof-Velocity 新增了哪些 reward 项? 这些新增项分别面向什么需求?

[INFO] Reward Manager: <RewardManager> contains 19 active terms.

Active Reward Terms		
Index	Name	Weight
0	track_lin_vel_xy	1.0
1	track_ang_vel_z	0.5
2	alive	0.15
3	base_linear_velocity	-2.0
4	base_angular_velocity	-0.05
5	joint_vel	-0.001
6	joint_acc	-2.5e-07
7	action_rate	-0.05
8	dof_pos_limits	-5.0
9	energy	-2e-05
10	joint_deviation_arms	-0.1
11	joint_deviation_waists	-1
12	joint_deviation_legs	-1.0
13	flat_orientation_l2	-5.0
14	base_height	-10
15	gait	0.5
16	feet_slide	-0.2
17	feet_clearance	1.0
18	undesired_contacts	-1

Unitree-G1-29dof-Velocity

[INFO] Reward Manager: <RewardManager> contains 7 active terms.

Active Reward Terms		
Index	Name	Weight
0	progress	1.0
1	alive	2.0
2	upright	0.1
3	move_to_target	0.5
4	action_l2	-0.01
5	energy	-0.005
6	joint_pos_limits	-0.25

Isaac-Humanoid-v0

对比 Isaac-Humanoid-v0 (Flat)，Unitree-G1-29dof-Velocity 的 RewardsCfg 扩展到 19 项，有更新也有新增。其中新增项如下，

新增 reward 项	解决问题
joint_vel	平滑关节速度，权重-0.001，用于平滑动作
base_height	基座保持 0.78m 高度，权重-10，稳定基座
gait	周期 0.8s、左右脚错半拍，权重 0.5，奖励好步态
foot_clearance	摆动腿抬到 0.1m，权重 1.0，奖励好步态
undesired_contacts	惩罚除踝关节外的其它部位接触地面，权重-1，防止蹭地等异常步态

- 训练输出中有两个 Curriculum 指标：Curriculum/terrain_levels 和 Curriculum/lin_vel_cmd_levels，它们各自含义是什么？为什么训练初期它们的值都很低？

Curriculum/terrain_levels：地形难度等级。等级越高代表当前 episode 采样到的

地形越复杂。

Curriculum/lin_vel_cmd_levels: 线速度命令难度等级。等级越高代表命令幅值越大、覆盖范围越广。

训练初期值都很低的原因：课程学习（Curriculum Learning）遵循“由易到难”的原则。先让机器人在最简单条件（平地、低速）下学会基本站立和行走技能，然后再逐步增加难度。如果一开始就给予复杂地形和高速指令，机器人会因任务过难而无法学到任何有效策略。

- 训练初期 Episode_Termination/bad_orientation 的值为 1.0，这说明了什么？随着训练推进，这个值应该发生什么变化？

bad_orientation 是一个终止条件，源码中以机器人躯干的倾斜角度超过“limit_angle=0.8rad”为判定触发。训练初期值为 1.0，表明几乎所有环境中的机器人都会因为跌倒而触发该终止条件。随着训练推进，该值应逐步下降并趋近于 0，当策略学会维持平衡后，机器人能够在整个 episode 期间保持直立，不会触发 bad_orientation。

1.3 奖励函数源码分析

操作手册中给出了 3 个奖励函数的源码（upward、energy、foot_clearance_reward），回答以下问题：

- 选择其中 **1** 个函数，用自己的话解释它的工作原理（输入是什么、计算逻辑是什么、输出代表什么）；
选择 upward 奖励函数。

```
Windsurf: Refactor | Explain | X
59 def upward(env: ManagerBasedRLEnv, asset_cfg: SceneEntityCfg = SceneEntityCfg
   ("robot")) -> torch.Tensor:
60     """Penalize z-axis base linear velocity using L2 squared kernel."""
61     # extract the used quantities (to enable type-hinting)
62     asset: RigidObject = env.scene[asset_cfg.name]
63     reward = torch.square(1 - asset.data.projected_gravity_b[:, 2])
64     return reward
```

输入：env 类型 ManagerBasedRLEnv，为机器人运行时环境信息；asset_cfg 类型 SceneEntityCfg，指定取哪个刚体，默认 root。

计算逻辑：按 asset_cfg.name 从 env.scene 里取出机器人刚体，读到重力方向投影向量（世界坐标系下的重力向量在机器人基座坐标系中的投影），再计算 reward（计算投影与基座 z 轴，即机器人期望的“向上”方向之间的点积）。当机器人完全直立时，重力全部投影到 z 轴负方向，点积最大；当机器人倾斜时，

重力投影偏离 z 轴，点积减小。

输出：一个张量(num_envs,)，代表每个并行环境在该步的“基座竖直程度”标量。

- foot_clearance_reward 中使用了 tanh 函数，它的作用是什么？为什么脚着地时不会误罚支撑脚？

tanh 相当于判别摆动相和支撑相。摆动腿的水平速度大，tanh 接近 1；支撑脚几乎静止， $\tanh \approx 0$ 。

tanh 使得在摆动相时才考虑抬腿高度，但对支撑相直接放行，所以脚着地时不会误罚支撑脚。

1.4 Sim2Sim 概念理解

- 用一段话解释：什么是 Sim2Sim？为什么不直接把 IsaacLab 训练的模型部署到真机上，而要先在 MuJoCo 中验证？

Sim2Sim 是指在将强化学习策略部署到真机之前，先在另一个独立的仿真环境中进行验证和测试的过程。Unitree RL Lab 的工作流程中，策略首先在 Isaac Lab 中训练，训练完成后，将模型导出为 ONNX 格式，然后在 MuJoCo 仿真器中运行 C++ 控制器。

先在 MuJoCo 中验证有以下几个原因：

1. 物理引擎差异：PhysX 与真机在接触模型、摩擦、阻尼、传感噪声等方面存在系统性偏差，直接部署可能瞬间失稳摔倒；
 2. 控制栈差异：Isaac Lab 训练是 Python+GPU 并行，真机是 C++ 实时控制 + DDS 通讯；
 3. 试错成本：真机摔倒会损坏关节/外壳，MuJoCo 可以反复重置。
- 用一段话解释：为什么 Sim2Sim 验证选择 MuJoCo 而不是 Gazebo？从物理引擎的接触模型、运行速度、API 设计等角度分析；
 1. 接触模型：MuJoCo 用解析式软接触+凸优化求解 (CG/Newton)，处理双足/四足这种频繁接触切换的高刚度场景仍稳定；Gazebo 默认 ODE/Bullet 是脉冲式硬接触，容易出现脚抖、穿透或数值发散。
 2. 运行速度：MuJoCo 是纯 C 实现、单线程就跑得很快，1kHz 控制环路轻松实时甚至超实时；Gazebo 强耦合 ROS、可视化与插件系统，相同模型下帧率明显更低。
 3. API 设计：MuJoCo 提供轻量 C/Python API，unitree_mujoco 直接通过 DDS 发布与真机一致的关节状态/命令话题，g1_ctrl 这套 C++ 控制代码不改一行就能在仿真和真机间切换；Gazebo 必须经 ROS topic 桥接，链路长、延迟难控且 ROS1/2 版本耦合。

- ONNX 格式的作用是什么？为什么不直接用 PyTorch 的.pt 文件进行部署？

ONNX (Open Neural Network Exchange) 是一种跨框架、跨平台的神经网络中间表示。把 PyTorch 训练得到的 policy 导出为 ONNX 后，可用 ONNX Runtime/Tensor RT 等轻量推理引擎在 C++端以毫秒级延迟运行，不再依赖 Python 解释器和 PyTorch 运行时。

不直接用.pt 的原因：.pt 是 PyTorch 序列化的 Python 对象，每次调用都需要 Python 运行时环境、PyTorch 库、Python-C++ 通信开销，难以满足机器人控制对低延迟、低资源的要求。

1.5Aelos 机器人基础操作

- 阅读 SMART 基础使用教程，了解 Aelos 机器人的启动流程和安全注意事项；
- 安装示教软件，完成与机器人的连接；
- 启动机器人，通过示教软件完成至少 2 个简单的动作（如站立、挥手、蹲下等）；
- 提供以下验证材料：

- 机器人启动截图 1 张；

参见：[机器人启动.png](#)

- 示教软件界面截图 1 张；

参见：[示教软件界面.png](#)

- 机器人执行动作的照片。

参见：[机器人执行动作.png](#)

注：因时间不足，未完全完成 Aelos 机器人的基础操作，截图会再更新。

1.6Aelos 机器人自由度分析

- 分析 Aelos 的自由度 (DoF) 分布；

一共 17 个自由度，

1. 头部：1 个；
2. 手臂：每只手臂 3 个自由度，共 6 个；
3. 腿部：每条腿 5 个自由度，共 10 个。

- 基于 Aelos 的自由度分析，思考以下问题：

- 你想利用 Aelos 完成哪些类型的任务？举例说明；

1. 运动类：比如跳舞、踢球等

2. 工程类：比如搬运小型物件
 3. 交互类：比如通过语音让机器人完成简单动作
- 你觉得 Aelos 目前最难以完成的任务是什么？为什么？

作为示教型机器人，

1. 手臂只有 3DoF 且没有手指、手腕，无法完成精细抓取操作；
2. 缺少高带宽 IMU、关节力矩反馈和可背驱舵机，无法快速奔跑、跳跃等；
3. 计算能力薄弱，难以应对突发变化。